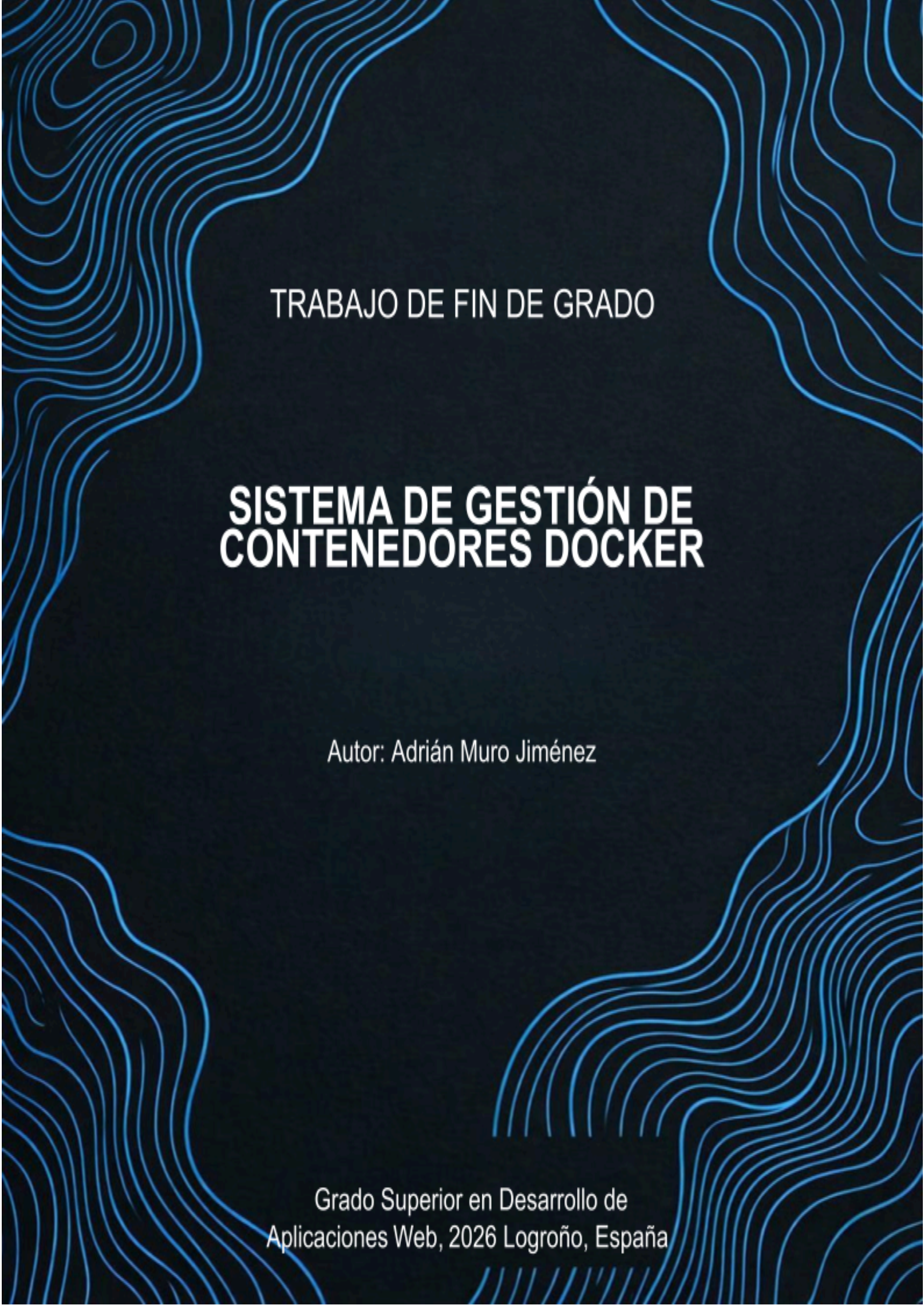




TRABAJO DE FIN DE GRADO

SISTEMA DE GESTIÓN DE CONTENEDORES DOCKER

Autor: Adrián Muro Jiménez

Grado Superior en Desarrollo de
Aplicaciones Web, 2026 Logroño, España

1. Introducción	3
2. Planificación del proyecto	4
3. Motivación del proyecto	5
4. Objetivos del proyecto	5
4.1 Objetivo general	6
4.2 Objetivos específicos	6
5. Requisitos del sistema	6
5.1 Distribuciones soportadas	6
5.2 Programas necesarios	6
6. Módulos utilizados	7
7. Funcionalidades de la aplicación	7
7.1 Funcionalidades de la API	7
7.2 Información del sistema	8
7.3 Gestión de usuarios	8
7.4 Acceso de usuarios	8
8. Funcionalidades de la interfaz web	9
8.1 Representación del hardware e información del sistema	9
8.2 Notificación y confirmación	9
9. Diseño de la aplicación	10
9.1 Vista de login	10
9.2 Vista de página de inicio	10
9.3 Vista de menú desplegable	11
9.4 Vista de configuración de usuario	11
9.5 Vista de logs del sistema	12
9.6 Vista de creación de usuario	12
9.7 Vista de creación de contenedor	13
9.8 Vista de información del sistema	14
9.9 Vista de edición del contenedor	15
9.10 Vista de gestor de archivos	16
10. Diseño de la Base de datos	16
10.1 Inicial	16
10.2 Final	17
11. Diseño de la API	17
11.1 Listado del hardware	17
11.2 Gestión del estado de los contenedores	18
11.3 Creación de nuevos servidores	18
11.4 Edición de contenedores	19
11.5 Gestión de archivos dentro del contenedor	19
12. Desarrollo	20
12.1 Backend	20
12.2 Frontend	21
13. Estudio de mercado	22
13.1 Visión general de los competidores	22
13.2 Comparativa funcional	23
13.3 Arquitectura técnica	23

13.4 Público objetivo	24
13.5 Diferenciadores clave	24
14. Conclusión	24
15. Bibliografía	25

1. Introducción

En este trabajo se desarrollará una aplicación web orientada a la gestión y monitorización de servidores de videojuegos y aplicaciones, basada en el uso de contenedores Docker sobre sistemas Linux. El objetivo principal es permitir la administración de estos servidores mediante una interfaz web intuitiva y amigable sin tener que recurrir a la consola de comandos.

Todo ello se realizará a través de una API que ejecutará las acciones con una interfaz web que enviará peticiones a la API dependiendo de las acciones que realice el usuario.

El acceso a la web se realizará mediante un sistema de autenticación diferenciando entre usuarios normales y administradores. En función del rol asignado, determinadas acciones críticas, como la creación de nuevos usuarios o configuración avanzada de contenedores, estarán restringidas exclusivamente a los administradores del sistema.

La aplicación hará uso de una base de datos para almacenar información relacionada con los usuarios, permisos, configuraciones de los contenedores, registros de actividad y otros datos necesarios para el correcto funcionamiento de la app. De forma complementaria, el entorno Docker y el sistema operativo actuarán como fuente de información en tiempo real, desde donde se obtendrán datos como el estado de los contenedores, el uso de recursos y los servicios activos.

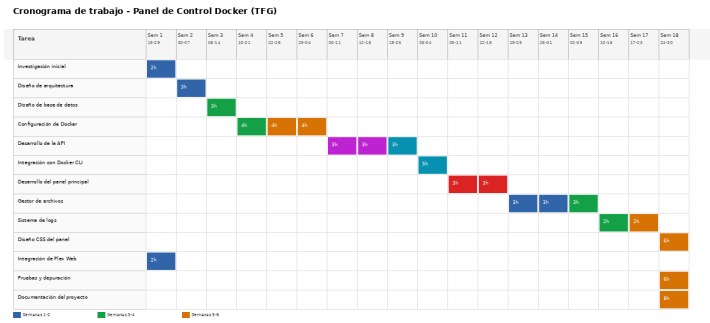
Para el desarrollo de la API se utilizará PHP, el cual actuará como intermediario entre la interfaz web y el motor de Docker. La comunicación con Docker se realiza mediante una API interna desarrollada en PHP, la cual recibe peticiones desde la interfaz web y ejecuta comandos Docker en el sistema mediante la función `shell_exec()` la cual permite ejecutar comandos en el sistema desde PHP para crear, gestionar y supervisar contenedores. Esta API propia permite estructurar las operaciones de forma controlada y devolver respuestas en formato JSON al frontend.

La interfaz web está construida utilizando HTML y CSS, incorporando JavaScript únicamente para proporcionar interactividad, validación de formularios y comunicación asíncrona con la API mediante peticiones AJAX.

Adicionalmente, se empleará Docker y entornos virtuales con el fin de aislar la aplicación del sistema anfitrión, evitar la instalación directa de dependencias y garantizar un funcionamiento estable, reproducible y seguro.

2. Planificación del proyecto

Actividad	Descripción	Tiempo estimado
Investigación inicial	Revisión de Docker, contenedores, paneles de control, APIs en PHP	2h
Diseño de arquitectura	Estructura del panel, rutas, módulos, base de datos	3 h
Diseño de base de datos	Tablas de contenedores, logs, usuarios	3 h
Configuración de Docker	Pruebas con contenedores (Plex, Minecraft, Python, Unturned)	14 h
Desarrollo de la API	actions.php, crear.php, editar.php, inspect, delete	10 h
Integración con Docker CLI	Comandos start/stop/restart/inspect/cp/exec	5 h
Desarrollo del panel principal	Listado de contenedores, estados, botones de acción	6 h
Desarrollo del gestor de archivos	listar, subir, borrar, crear carpetas, descargar	10 h
Sistema de logs	Registro de acciones, filtros, paginación	5 h
Diseño CSS del panel	Temas, tarjetas, modales, responsive	6 h
Integración de Plex Web	Generación de URL, comprobación de estado	2 h
Pruebas y depuración	Test de API, Docker, filemanager, errores	6 h
Documentación del proyecto	Memoria, diagramas, casos de uso, API	8 h



3. Motivación del proyecto

La motivación de este proyecto surge tras haber utilizado durante años diferentes paneles de gestión de servidores como **CubeCoders (AMP)** o **Pterodactyl**. Siempre me llamó la atención la complejidad interna que tenían y, al mismo tiempo, lo sencillo que parecía utilizarlos desde fuera. Eso despertó en mí la curiosidad por entender cómo funcionaban realmente y qué había detrás de cada acción que el usuario ejecutaba. Con el tiempo descubrí que, aunque desde fuera parecían herramientas fáciles de replicar, su desarrollo implicaba una arquitectura mucho más profunda de lo que imaginaba.

Por otro lado, la motivación principal nace del deseo de crear **mi propio panel personal**, completamente adaptado a mis necesidades y sin depender de limitaciones externas. La mayoría de los paneles existentes imponen restricciones en cuanto al número de contenedores, tipos de servicios o incluso requieren **suscripciones mensuales** para desbloquear funciones avanzadas. Mi objetivo era justo lo contrario: disponer de una herramienta **totalmente libre**, sin límites y que pudiera integrar cualquier aplicación basada en contenedores que yo quisiera utilizar.

4. Objetivos del proyecto

El objetivo principal de este proyecto es desarrollar un panel web propio capaz de gestionar contenedores Docker de manera sencilla, flexible y sin las limitaciones habituales de otros paneles comerciales. A partir de esta idea central, se establecen los siguientes objetivos específicos:

4.1 Objetivo general

Diseñar e implementar un panel de control completo que permita crear, gestionar, supervisar y administrar contenedores Docker mediante una API propia desarrollada en PHP, ofreciendo una alternativa ligera, personalizable y sin restricciones de uso ni costes asociados.

4.2 Objetivos específicos

1. Crear una API modular para controlar Docker
2. Desarrollar un panel web intuitivo y responsive
3. Implementar un gestor de archivos dentro del contenedor
4. Integrar un sistema de logs
5. Diseñar una base de datos funcional y optimizada
6. Permitir la integración de aplicaciones externas
7. Garantizar la independencia del usuario

5. Requisitos del sistema

A Continuación, se mostrarán los requisitos.

5.1 Distribuciones soportadas

La aplicación será compatible con las siguientes distribuciones:

- **Familia Debian:** Debian, Ubuntu, Mint... (EN PRUEBAS)
- **Familia Windows:** 11 y 10

5.2 Programas necesarios

La aplicación hace uso de varios programas para funcionar:

- MySQL
- PHP
- Docker
- Apache

6. Módulos utilizados

- **Desarrollo web en entorno cliente:** Se usaron los conocimientos de esta asignatura para desarrollar la interfaz ya que se usarán lenguajes de programación web orientados al cliente.
- **Desarrollo web en entorno servidor:** Se usaron los conocimientos de esta asignatura para desarrollar la interfaz ya que se usarán lenguajes de programación web orientados al servidor.
- **Diseño de interfaces web:** Se planteo y se maqueto una interfaz web para que el usuario interactúe con el programa por lo que también se hizo uso de conocimientos de esta asignatura.
- **Despliegue de aplicaciones web:** Las dos aplicaciones se van a desplegar en un servidor a través de scripts, servicios y contenedores por lo que varios conceptos de esta asignatura se tuvieron en cuenta.
- **Programación:** Se usaron conocimientos recibidos en esa asignatura para programar la aplicación ya que son conocimientos básicos y fundamentales para la programación.
- **Sistemas informáticos:** El programa ejecuta comandos de la shell y hace uso de scripts en bash para configurar la aplicación en el sistema. Además de que la aplicación se ejecuta en un sistema Linux.

7. Funcionalidades de la aplicación

La aplicación está diseñada para ofrecer un sistema completo de gestión de servidores basados en contenedores Docker.

7.1 Funcionalidades de la API

A continuación, se describen las funcionalidades principales que ofrece:

- Creación de contenedores
- Arranque y parada de contenedores
- Eliminación de contenedores
- Gestión de archivos del servidor
- Registro de actividad
- Comunicación asíncrona con el frontend

La API obtiene los siguientes elementos hardware: la CPU y la RAM.

- **CPU:** Se mostrará información básica del procesador, como datos sobre el uso de la CPU en tiempo real, permitiendo conocer la carga del sistema en estos contenedores.
- **RAM:** Se mostrará la cantidad total de memoria RAM disponible, así como el uso actual de la memoria, indicando la memoria utilizada y libre. Esta información permitirá comprobar si el servidor dispone de recursos suficientes para los servicios en ejecución.
- **DISCO:** Se mostrará información sobre el almacenamiento disponible en el sistema, incluyendo la capacidad total.

7.2 Información del sistema

La API obtiene información sobre el sistema operativo instalado, se obtiene la siguiente información:

- nombre de la distribución,
- versión numérica,
- nombre de la versión (en caso de que tenga),
- familia de la distribución,
- versión de kernel y ip del host.

7.3 Gestión de usuarios

La API gestiona los usuarios del sistema, en caso de que el usuario con el que hemos accedido cuente con permisos de admin. Se podrán ejecutar las siguientes opciones: añadir un nuevo usuario y ver logs de los demás usuarios.

Al añadir y modificar usuarios se podrán establecer los siguientes parámetros del usuario: nombre, contraseña y correo electrónico.

7.4 Acceso de usuarios

Para utilizar la API se requiere de una autenticación mediante los usuarios del sistema. Como ya se mencionó hay acciones que se pueden realizar dependiendo de los permisos con los que cuente el usuario que se esté usando.

8. Funcionalidades de la interfaz web

La interfaz web se encarga de visualizar e interactuar con los datos proporcionados por la API, permitiendo a los usuarios administrar los servidores y contenedores de forma sencilla. Para facilitar la navegación entre los distintos recursos, se implementó una barra lateral que organizará las secciones principales del panel, como contenedores, logs, Estado del servidor, y Actualizaciones.

La interfaz es responsiva, adaptándose automáticamente tanto a pantallas de ordenador como a dispositivos móviles, garantizando así una experiencia de usuario consistente y cómoda en cualquier tipo de dispositivo.

8.1 Representación del hardware e información del sistema

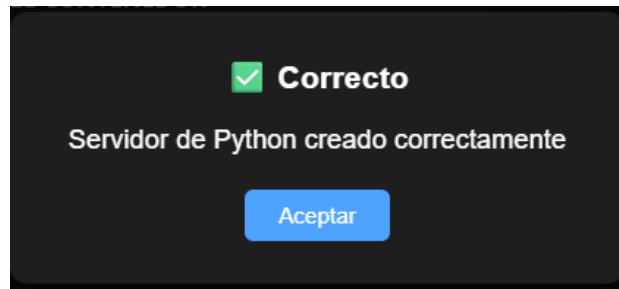
En una de las vistas de la aplicación se muestra el hardware y la información sobre el sistema. Esta vista se llama Estado del servidor, se muestran 7 recuadros en los que se representa la CPU, la GPU, la RAM, los Discos y la información del sistema:

- **Recuadro de la CPU:** Se mostrará el nombre de la CPU y en un gráfico y se representará su carga del sistema.
- **Recuadro de la GPU:** Se mostrará el nombre de la GPU y en dos gráficos se representarán su uso en tiempo real y la temperatura.
- **Recuadro de la RAM:** Se mostrará el uso en tiempo real de la RAM en un gráfico.
- **Recuadro de la información del sistema:** Se mostrará toda la información del sistema operativo en el recuadro. Se mostrará un icono de la distribución que se este usando.

8.2 Notificación y confirmación

Por último, la interfaz contará una unas ventanas emergentes para notificar el estado de una acción:

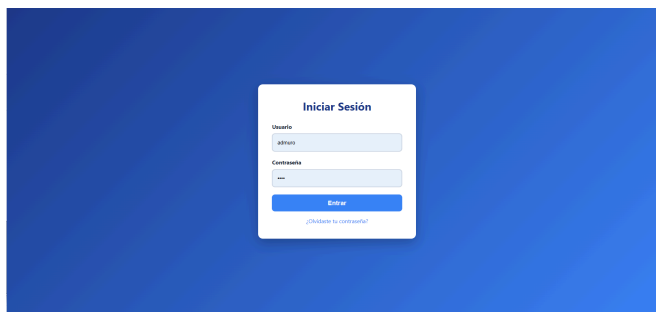
- **Confirmaciones:** Son ventanas emergentes que pedirán una confirmación, se mostrarán cuando se vaya a realizar un cambio como por ejemplo eliminar un contenedor.
- **Notificaciones:** Son ventanas emergentes que se mostrarán en el medio de la página para indicar si una acción se cumplió.



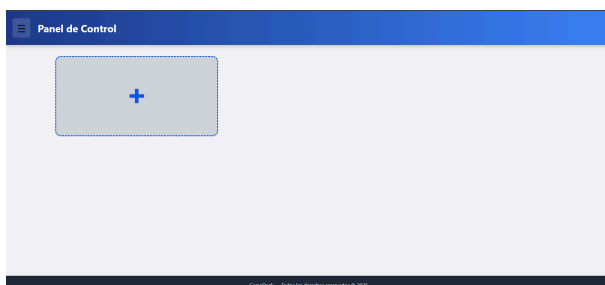
9. Diseño de la aplicación

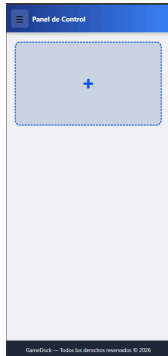
En este apartado mostraré la maqueta de la interfaz web.
En cada vista mostraré cómo se verá en pantallas horizontales (ordenadores y móviles).

9.1 Vista de login

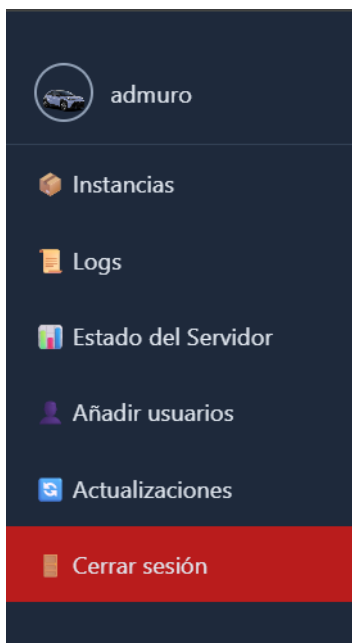


9.2 Vista de página de inicio

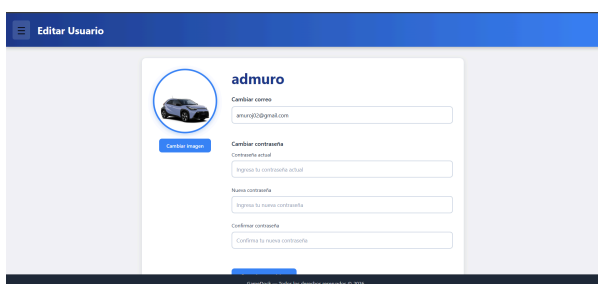




9.3 Vista de menú desplegable



9.4 Vista de configuración de usuario



Editar Usuario



Cambiar imagen

admuro

Cambiar correo

amuro02@gmail.com

Cambiar contraseña

Contraseña actual

Ingresar tu contraseña actual

Nueva contraseña

Ingresar tu nueva contraseña

Confirmar contraseña

Confirma tu nueva contraseña

Guardar cambios

Seleccionar tema

Claro

Oscuro

GameDock — Todos los derechos reservados © 2026

9.5 Vista de logs del sistema

Logs del Sistema

Historial de Actividad

Buscar...

dd/mm/aaaa

Todos los usuarios

Fecha	Usuario	Acción
2026-02-24 20:01:59	pepo	Inició sesión
2026-02-20 13:05:15	pruebaaa	Cambió su foto de perfil
2026-02-20 12:57:04	pruebaaa	Inició sesión
2026-02-20 12:56:25	pepe	Inició sesión
2026-02-20 12:56:22	pepe	Inició sesión
2026-02-20 12:55:00	pepe	Cerró sesión

Acción

Sistema

GameDock — Todos los derechos reservados © 2026

Logs del Sistema

Registro del Sistema

Buscar por usuario o acción

dd/mm/aaaa

Filtrar

FECHA	USUARIO	ACCIÓN
2026-04-20 07:55:58	admuro	Inició sesión
2026-04-17 10:58:02	admuro	Inició sesión
2026-04-17 10:05:47	admuro	Creó el servidor Unturned 'otidatd'
2026-04-17 10:05:00	admuro	Eliminó el contenedor Python 'dandada'
2026-04-17 10:04:56	admuro	Creó el contenedor Python 'dandada'
2026-04-17 10:03:37	admuro	Eliminó el contenedor Python 'prueba'
2026-04-17 10:03:34	admuro	Creó el contenedor Python 'prueba'
2026-04-17 10:00:38	admuro	Creó el contenedor Python 'distatd'
2026-04-17 09:59:43	admuro	Creó el contenedor Python 'hyperbarr'
2026-04-17 09:58:04	admuro	Creó el contenedor Python 'tiffy'

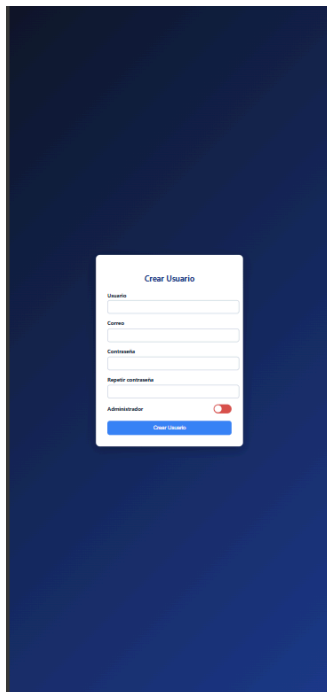
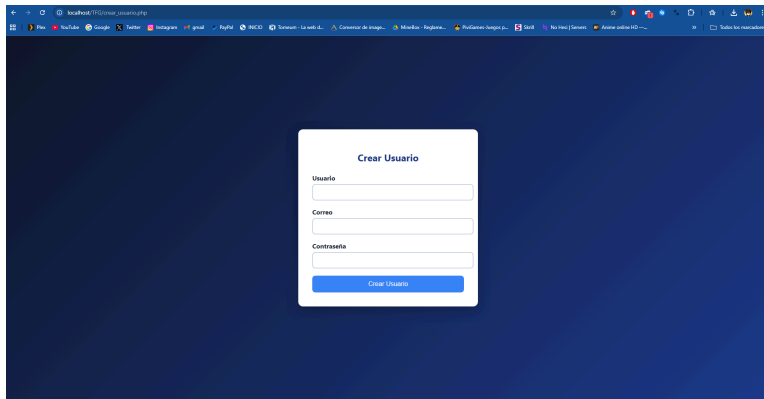
Página 1 de 57

Siguiente

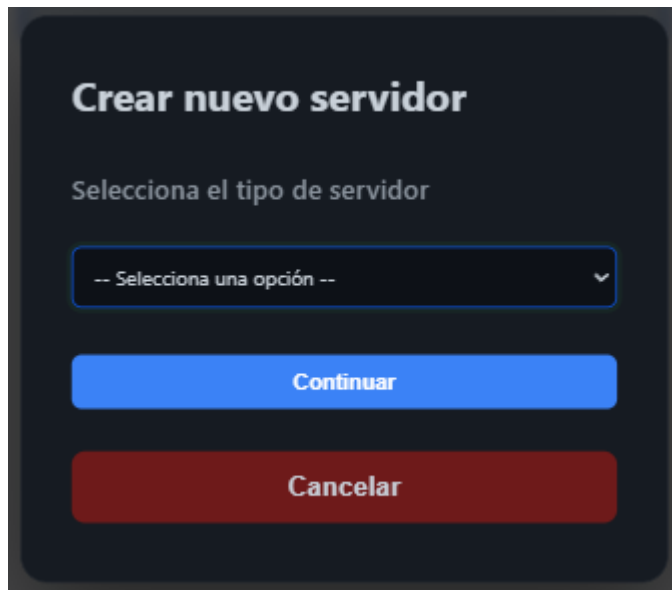
Última

GameDock — Todos los derechos reservados © 2026

9.6 Vista de creación de usuario



9.7 Vista de creación de contenedor



Crear nuevo servidor

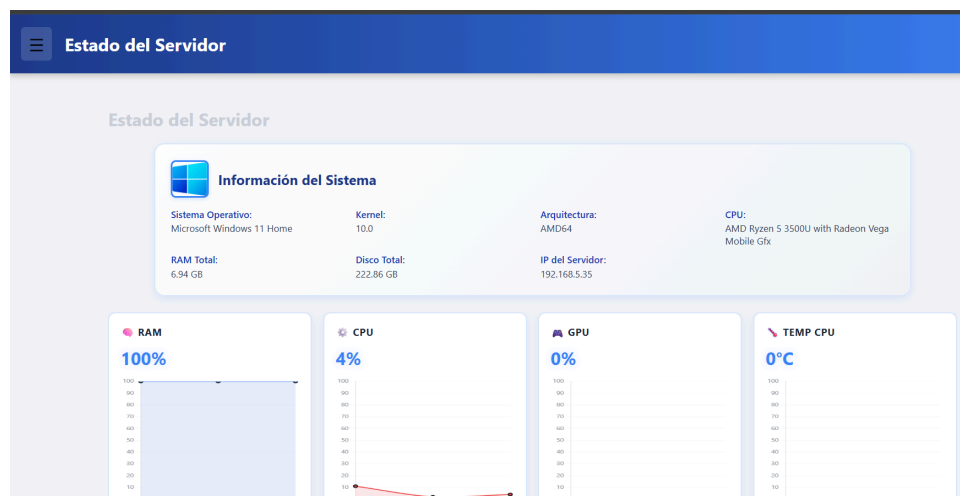
Selecciona el tipo de servidor

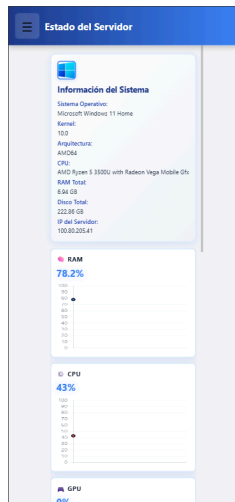
-- Selecciona una opción --

Continuar

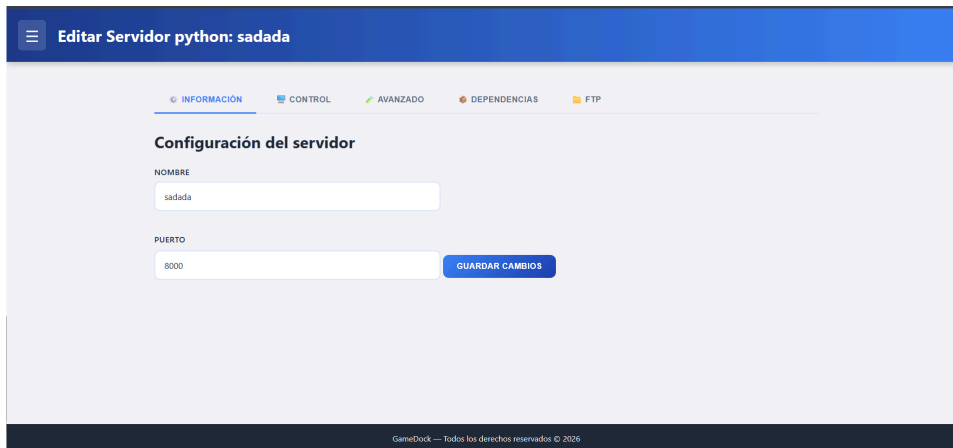
Cancelar

9.8 Vista de información del sistema





9.9 Vista de edición del contenedor



Editar Servidor python: sadada

INFORMACIÓN CONTROL AVANZADO DEPENDENCIAS FTP

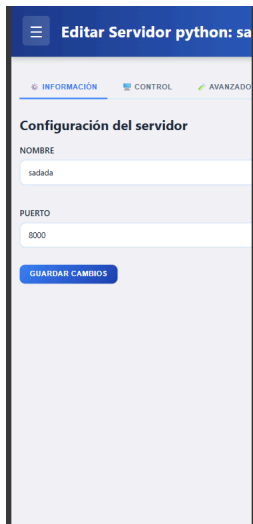
Configuración del servidor

NOMBRE
sadada

PUERTO
8000

GUARDAR CAMBIOS

GameDock — Todos los derechos reservados © 2020



Editar Servidor python: sa

INFORMACIÓN CONTROL AVANZADO

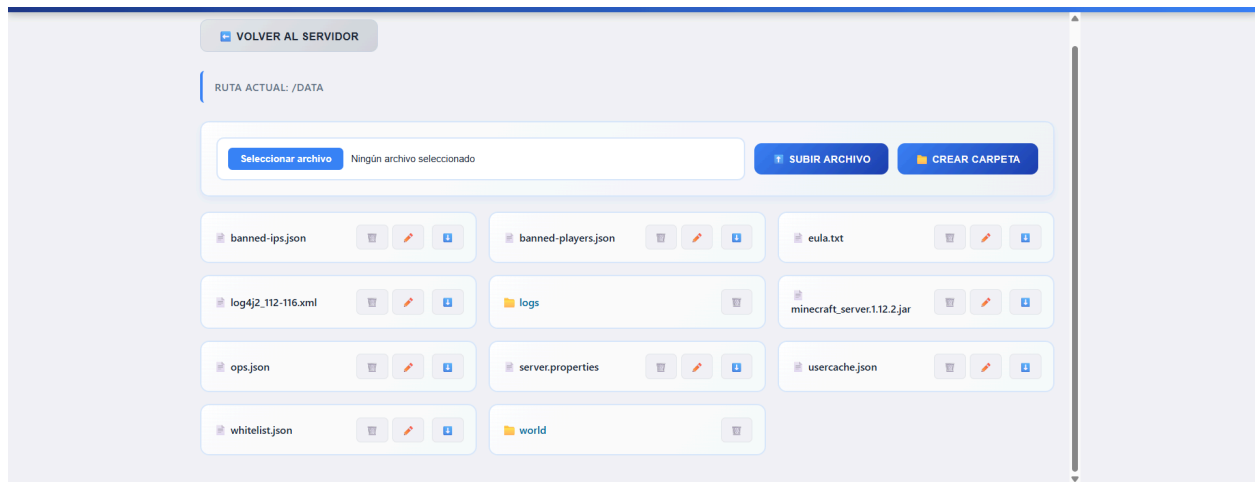
Configuración del servidor

NOMBRE
sadada

PUERTO
8000

GUARDAR CAMBIOS

9.10 Vista de gestor de archivos

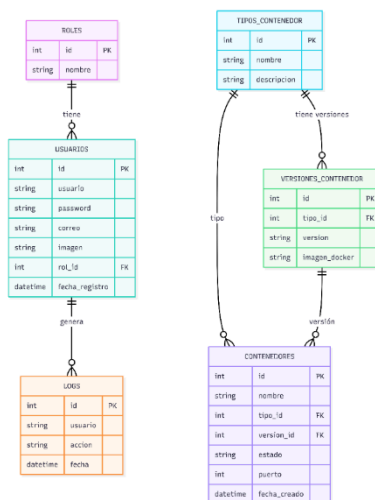


10. Diseño de la Base de datos

A Continuación, se mostrarán los requisitos.

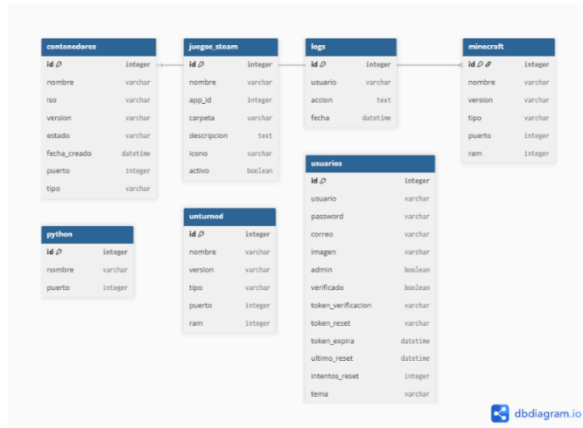
10.1 Inicial

Como se puede observar en la imagen, este es el modelo de base de datos básico que utilicé al inicio del proyecto. Contaba con una estructura inicial que servía como punto de partida; sin embargo, con el desarrollo del sistema me di cuenta de que no resultaba lo suficientemente flexible y que las relaciones entre las entidades no eran las más adecuadas. Por este motivo, opté por rediseñar el modelo y adoptar una nueva estructura, que se presenta más adelante, con el objetivo de mejorar la organización de los datos y la eficiencia del sistema.



10.2 Final

En este nuevo modelo se puede observar cómo se crea una tabla para cada tipo de contenedor. Si bien es cierto que esto implica la generación de un mayor número de tablas, también permite una mayor flexibilidad, ya que cada contenedor puede almacenar distintos tipos de datos en función de las necesidades específicas. Por otro lado, se intentó implementar un contenedor global para cada tipo de juego utilizando SteamCMD; sin embargo, su desarrollo resultó complejo, por lo que finalmente se optó por crear una imagen independiente para cada tipo de contenedor. Esta solución facilitó la gestión y el despliegue de los mismos.



11. Diseño de la API

Definiré los métodos de la API:

11.1 Listado del hardware

GET /api/estado.php → Obtendrá toda la información de la máquina en la cual se ejecuta la aplicación

```

> fetch("../api/estado.php")
  .then(r => r.json())
  .then(data => console.log(data));
< ▶ Promise {<pending>}

{
  sistema: { ... }, ram: { ... }, cpu: { ... }, gpu: { ... }, disco: { ... }, ... }
  ▶ cpu: { porcentaje: '9', temperatura: null }
  ▶ disco: { porcentaje: 78.8 }
  ▶ gpu: { porcentaje: 0, temperatura: 0 }
  ▶ ram: { total: 6.94, usada: 6.938984375, porcentaje: 100 }
  ▶ red: { total: 143 }
  ▶ sistema: { so: 'Microsoft Windows 11 Home', kernel: '10.0', arquitectura: 'AMD64', uptime: '00:00:00', cpu: 'AMD Ry
  ▶ [[Prototype]]: Object
  
```

11.2 Gestión del estado de los contenedores

POST /api/actions.php → Permite ejecutar acciones sobre un contenedor Docker.

Acciones disponibles:

- start → Inicia un contenedor
- stop → Detiene un contenedor
- restart → Reinicia un contenedor
- delete → Elimina un contenedor
- inspect → Obtiene su estado actual

```
fetch("api/python/actions/manage.php", {  
  method: "POST",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify({  
    tipo: "start",  
    nombre: "python_micontenedor"  
  })  
});
```

11.3 Creación de nuevos servidores

POST /api/crear.php → Crea un nuevo contenedor Docker del tipo seleccionado (Minecraft, Plex, Python, Unturned...).

```
fetch("api/python/create.php", {  
  method: "POST",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify({  
    nombre: "miapp",  
    puerto: 9000,  
    dependencias: "flask\nrequests"  
  })  
})  
.then(r => r.json())  
.then(res => console.log(res));
```

11.4 Edición de contenedores

POST /api/editar.php → Permite modificar la configuración del contenedor, como su nombre o parámetros internos.

Funciones:

- Renombrar contenedor
- Actualizar datos en la base de datos
- Validar existencia del contenedor

```
fetch("api/python/update.php", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    id: 12,
    nombreActual: "python_miapp",
    nuevoNombre: "python_miapp2",
    nuevoPuerto: 9001,
    puertoActual: 9000
  })
})
.then(r => r.json())
.then(res => console.log(res));
```

11.5 Gestión de archivos dentro del contenedor

GET /filemanager/api/listar.php → Lista los archivos y carpetas dentro del contenedor.

POST /filemanager/api/subir.php → Sube un archivo desde el navegador al contenedor.

POST /filemanager/api/borrar.php → Elimina un archivo o carpeta del contenedor.

GET /filemanager/api/descargar.php → Descarga un archivo desde el contenedor al equipo del usuario.

```
fetch("api/python/listar.php?servidor=python_miapp")
  .then(r => r.json())
  .then(res => console.log(res));
```

12. Desarrollo

A continuación, se describen las tecnologías empleadas tanto en el backend como en el frontend, así como las herramientas auxiliares utilizadas durante el desarrollo.

12.1 Backend

Lenguaje y entorno

El proyecto está desarrollado íntegramente en **PHP**, debido a su facilidad de despliegue, su integración nativa con servidores web como Apache y su capacidad para ejecutar comandos del sistema. Esto es fundamental, ya que el panel necesita interactuar directamente con Docker mediante:

- `shell_exec()`
- `exec()`
- `escapeshellarg()`

Estas funciones permiten ejecutar comandos como:

- `docker start`
- `docker stop`
- `docker restart`
- `docker inspect`
- `docker cp`
- `docker exec`

La API devuelve siempre respuestas en **formato JSON**, lo que facilita su uso desde el frontend.

API propia en PHP

En lugar de usar frameworks como FastAPI, Laravel o Symfony, se optó por una **API ligera y modular en PHP**, organizada en endpoints independientes:

- `/api/actions.php` → iniciar, detener, reiniciar, eliminar contenedores
- `/api/crear.php` → creación de nuevos servidores
- `/api/editar.php` → edición de contenedores
- `/filemanager/api/*` → gestión de archivos dentro del contenedor
-

La API está diseñada para ser:

- **simple**
- **rápida**
- **compatible con cualquier hosting**
- **sin dependencias externas**

MySQL / MariaDB

Se utiliza una base de datos MySQL para almacenar:

- Información de los contenedores
- Tipos de servidores
- Logs de acciones
- Datos del usuario

Docker Engine

Docker es el núcleo del proyecto. El panel no solo muestra información, sino que **controla Docker directamente**.

Se utilizan comandos como:

- docker run → creación de contenedores
- docker rm -f → eliminación
- docker inspect → estado
- docker exec → ejecutar comandos dentro del contenedor
- docker cp → copiar archivos

12.2 Frontend

HTML, CSS y JavaScript

El frontend está construido de forma nativa sin frameworks, lo que permite:

- mayor control visual
- menor peso
- carga más rápida
- integración directa con PHP

CSS personalizado

Se ha desarrollado un sistema de temas con:

- colores dinámicos
- estilos modulares
- componentes reutilizables
- diseño responsive

Incluye elementos como:

- tarjetas de contenedores
- gestor de archivos
- panel de logs
- modales
- botones de acción

JavaScript (Fetch API)

El frontend se comunica con la API mediante **fetch()**, enviando y recibiendo JSON.

Ejemplo real:

```
fetch("/TFG/contenedores/plex/api/actions.php", {  
  method: "POST",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify({ tipo: "start", nombre: "plex_servidor" })  
})  
.then(r => r.json())  
.then(res => console.log(res));
```

Esto permite:

- iniciar/stop/restart contenedores
- listar archivos
- subir/borrar contenido
- actualizar el panel en tiempo real

13. Estudio de mercado

El mercado objetivo de GameDock se sitúa en la intersección de tres sectores tecnológicos en crecimiento:

13.1 Visión general de los competidores

- CubeCoders AMP
Es una plataforma comercial orientada a la gestión de servidores de juegos y aplicaciones. Destaca por su estabilidad, soporte profesional y ecosistema de módulos.
- Pterodactyl
Es un panel open-source ampliamente utilizado para servidores de juegos. Su arquitectura se basa en contenedores aislados (Wings) y está orientado a comunidades y proveedores de hosting.
- GameDock
Es un panel modular basado en Docker, diseñado para gestionar servidores de juegos, contenedores Python, servicios multimedia (Plex) y otros entornos de forma unificada, con un enfoque en simplicidad, personalización y automatización.

13.2 Comparativa funcional

Plataforma	Enfoque principal
CubeCoders AMP	Hosting profesional y comercial de servidores de juegos
Pterodactyl	Gestión de servidores de juegos en entornos comunitarios o hosting
GameDock	Panel modular multi-servicio basado en Docker (juegos, Python, Plex, utilidades)

Conclusión: GameDock no compite directamente en el mismo enfoque; ofrece un enfoque más amplio y flexible.

13.3 Arquitectura técnica

Plataforma	Arquitectura
AMP	Sistema propio, no basado en Docker
Pterodactyl	Contenedores aislados mediante Wings, pero no Docker estándar
GameDock	Docker nativo, volúmenes, logs, dependencias, modularidad total

Ventaja de GameDock: El uso de Docker estándar permite compatibilidad universal, portabilidad y facilidad de mantenimiento.

13.4 Público objetivo

Plataforma	Arquitectura
AMP	Empresas de hosting, usuarios avanzados
Pterodactyl	Comunidades de juegos, hosting
GameDock	desarrolladores, administradores de juegos, usuarios medios

GameDock cubre un espectro más amplio, especialmente para quienes buscan simplicidad y control.

13.5 Diferenciadores clave

GameDock ofrece características que ninguno de los competidores combina:

- **Gestión Docker nativa**
- **Módulos multi-servicio (Python, Plex, juegos, utilidades)**
- **Temas dinámicos y personalización profunda**
- **Automatización avanzada (dependencias, backups, actualizaciones)**
- **Interfaz moderna y coherente**
- **Sistema de logs y auditoría integrado**
- **Arquitectura totalmente modular y extensible**

14. Conclusión

Puedo afirmar que este proyecto ha supuesto un desafío constante. En varias ocasiones he necesitado apoyo externo, tanto de amigos como de herramientas basadas en inteligencia artificial, para poder desarrollar determinadas funcionalidades. Esto se debe a que el proyecto planteado incluía aspectos que no habíamos abordado en clase y sobre los que, además, no siempre resultaba sencillo encontrar información en Internet.

Aun así, considero que ha sido una experiencia muy enriquecedora, ya que he adquirido numerosos conocimientos durante su desarrollo. Además, el proceso también ha resultado satisfactorio a nivel personal.

Por otro lado, he podido comprobar la utilidad de la herramienta desarrollada, ya que responde a necesidades reales. Anteriormente utilizaba servicios similares de pago, pero la principal ventaja de esta solución es que puedo adaptarla completamente a mis necesidades, personalizándola y ampliándola según sea necesario.

15. Bibliografía

Investigación sobre la creación de APIs y Dockerfiles

Docker: <https://docs.docker.com/>

ISO de minecraft: <https://github.com/itzg/docker-minecraft-server?tab=readme-ov-file>

PLEX: <https://watch.plex.tv/es/me>

Shell exec: <https://www.php.net/manual/en/function.shell-exec.php>

Fetch Api: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

Investigación para desarrollar diversas funcionalidades e información

Copilot: <https://copilot.microsoft.com/>

Claude AI: <https://claude.ai>

Imágenes Docker:
https://aulavirtual-educacion.larioja.org/pluginfile.php/1166725/mod_resource/content/15/site/index.html

Inspiraciones para la interfaz web

Cubecoders: <https://cubecoders.com/>

Pterodactyl: <https://pterodactyl.io/>